Kiosk App — 개발자 인수인계

witworldwide 관광 안내 키오스크 · 저장소 `wit-project-sku/kiosk-electron` · v5.0.27 · 2026-10-21 기준

1. 어떤 앱인가
2. 내 PC에서 실행하기
3. 폴더 구조
4. 앱 동작 구조
5. 설치 장소와 kioskId
6. 데이터: 출처, 가져오는 시점, 저장 위치
7. 구글 시트 상세
8. 키오스크 PC의 파일 위치
9. 매일 전원 스케줄 (자동 종료)
10. 배포와 자동 업데이트
11. .env 설명
12. 자주 하는 작업
13. 주의할 점

1. 어떤 앱인가

관광지용 터치스크린 안내 키오스크입니다 (Electron 34 + React 19 + TypeScript, CSS Modules). Windows 전용입니다. **하나의 빌드가 모든 장소에서 실행됩니다.** 각 PC에는 자기가 어느 장소인지(**kioskId**, 예: `W004`)가 설정되어 있고, 이 값에 따라 화면, 콘텐츠, 기능(카드 단말기, 카메라, 게임 등)이 결정됩니다.

주요 기능: 안내 화면(상점, 행사, 궁궐, 교통, 택스리펀, 환율, 날씨, 제주 항공편/여객선), AI 코스 추천, QR로 받는 AR 한복 포토부스, 카드 결제가 되는 기부/상점 웹페이지, 유동인구 카운팅, 제주 모션 게임, 키 측정(제주, ZED 카메라).

2. 내 PC에서 실행하기

1. Node 20 이상 설치 후 `npm install` (Electron용 네이티브 모듈도 함께 다시 빌드됩니다).
2. 실제 `.env`와 `secrets/service-account.json`, `secrets/service-account-osaek.json`은 이전 개발자에게 직접 받습니다 (git에 없음). 모든 키 목록은 `.env.example`에 있습니다.
3. `.env`에 `DEV_MODE=true`를 넣으면 홈 화면에 장소 전환 버튼이 나타나서 모든 키오스크 화면(W001~W202)을 볼 수 있습니다.
4. `npm run dev` → 핫 리로드로 실행. `F12`로 개발자 도구를 엽니다.

명령어	하는 일
`npm run dev`	개발 모드 실행
`npm run typecheck` / `npm run lint`	커밋 전 검사
`npm run sync:sheet`	구글 시트를 받아 `src/renderer/src/data/*.generated.ts` 재생성 (7장 참고)
`npm run build:win`	운영용 설치 파일 생성 (패치 버전 자동 증가)
`npm run build:win:beta`	베타 설치 파일 (운영 버전과 같은 PC에 나란히 설치 가능)

3. 폴더 구조

```
kiosk-app/
├─ src/
│  ├─ main/                ← 백엔드 (Electron 메인 프로세스, Node). 로직·네트워크·DB 전부 여기
│  │  ├─ index.ts          시작 순서: DB → 서비스 → 전원 → 창 → 동기화/업데이트
│  │  ├─ container.ts      모든 서비스를 생성 (어떤 서비스가 있는지 한눈에)
│  │  ├─ core/             앱 공통 기반
│  │  │    KioskConfigStore.ts   kioskId 읽기/쓰기 (kiosk-config.json)
│  │  │    AppConfigStore.ts     앱 설정           (settings.json)
│  │  │    PowerManager.ts       매일 재부팅 / 종료 / 자동 시작
│  │  │    env.ts  .env 로딩       paths.ts  모든 디스크 경로
│  │  │    PaymentAgentManager.ts  카드 결제 에이전트 실행
│  │  │    ZedSidecarManager.ts    제주 키 측정 카메라(Python) 실행
│  │  │    logger.ts             로그 파일
│  │  ├─ services/         기능별 서비스: ShopService, BannerService, WeatherService,
│  │  │    ExchangeService, FlightService, SailingService, OutfitService, StatsService …
│  │  │    sync/           야간 동기화 + 구글 시트 (SyncService, GoogleSheetsSyncTransport)
│  │  │    photo/ ai/ camera/  AR 한복 사진       footfall/  유동인구 카운팅
│  │  │    height/ motion/ spotDiff/ drive/  제주 기능, 게임, 구글 드라이브 업로드
│  │  ├─ database/         SQLite: Database.ts, migrations/ (001~008), repositories/
│  │  ├─ ipc/handlers/     UI가 호출하는 API (기능별 파일: shop.handlers.ts …)
│  │  ├─ updater/          자동 업데이트 + 원격 "지금 업데이트" 명령
│  │  └─ windows/          메인 창 + 고객용 디스플레이 창
│  ├─ preload/             안전한 연결 통로: 메인의 IPC API를 UI에 window.api로 노출
│  ├─ renderer/src/        ← 프론트엔드 기본 틀 (React)
│  │    App.tsx            kioskId에 맞는 레이아웃 선택
│  │    store/             Zustand 스토어 (shopStore, weatherStore, languageStore …)
│  │    hooks/             useKioskController (화면 이동 + 무입력 초기화), usePhotoWorkflow …
│  │    lib/               작은 도우미 (i18n, shops, videoMap …)
│  │    data/              구글 시트로 만든 *.generated.ts + sheet-cache/ (원본 CSV)
│  ├─ layouts/             ← 실제 화면. 장소 계열별 폴더
│  │    InsadongLayout.tsx  insadong/   (W001, W002 / W003은 NamInsadongLayout)
│  │    OsanLayout.tsx      osan/       (W004 오색시장)
│  │    HwaseongLayout.tsx  hwaseong/   (W005 화성휴게소)
│  │    JejuLayout.tsx      jeju/       (W006 공항, W007 여객터미널, W008 세계자연유산)
│  │    KadaLayout.tsx      kada/       (W202 베트남)
│  │    photo/  공용 AR 사진 화면       components/  공용 위젯
│  └─ shared/              양쪽(메인/화면)에서 함께 사용
│       config/kioskLocations.ts   ★ 전체 장소 목록과 장소별 기능
│       types/  constants/  ipc/   (IPC 채널 이름 + 계약)
├─ themes/                장소별 색상/폰트 (JSON)
├─ resources/             mediapipe (카메라 AI 모델), videos (개발용), 도움말 이미지
├─ payment-agent/         컴파일된 카드 결제 서버 (별도 저장소, 여기서 수정 금지)
├─ zed-height/            ZED 키 측정 카메라용 Python 스크립트 (제주)
├─ scripts/               빌드, sync-sheet, 프로비저닝, 자체 테스트, db-inspect
├─ tools/location-tester/   설치된 키오스크의 장소 변경 도구
├─ secrets/               구글 서비스 계정 JSON (gitignore 대상)
├─ docs/                  AUTO_UPDATE.md, FOOTFALL.md, UPDATE_COMMAND_API.md, 이 문서
├─ provision-kiosk.ps1    PC의 kioskId 설정
├─ Set/UnsetAutoShutdown*.bat  예전 수동 전원 스크립트 (지금은 앱이 직접 처리, 9장)
└─ electron-builder.yml   설치 파일 설정 (베타는 .beta.yml)
```

기억할 규칙: 데이터, 네트워크, 파일, 타이머 → `src/main/services`. 방문객이 보는 화면 → `src/layouts/<장소>`. 화면(UI)은 인터넷을 직접 호출하지 않고, `window.api`(IPC)를 통해 메인에 요청합니다.

4. 앱 동작 구조

```
구글 시트 / witteria API / 날씨 / 항공편
          │  (가져옴)
          ▼
  src/main/services/*  ──저장──▶  SQLite  (%APPDATA%\kiosk-app\data\kiosk.db)
          │
  src/main/ipc/handlers/*   (메인이 요청에 응답하고 "변경됨" 이벤트를 보냄)
          │
  src/preload  → window.api
          │
  화면 스토어 (Zustand)  →  src/layouts/<장소>/*.tsx  (화면)
```

- **시작 순서** (`src/main/index.ts`): .env 로딩 → DB 열기 + 마이그레이션 → 서비스 생성 → 전원 스케줄 설정 → 창 열기(저장된 데이터로 바로 표시) → 날씨/항공편/환율 주기 조회 시작 → 결제 에이전트 시작(카드 단말기 있는 장소만) → 자동 업데이트 시작 → 백그라운드에서 콘텐츠 새로고침.
- **화면 크기**: 모든 레이아웃은 고정 **2160 × 3840** 캔버스를 모니터에 맞게 확대/축소합니다. 그래서 Figma px = CSS px 입니다.
- **화면 이동과 무입력**: `useKioskController.ts`. 3분 동안 터치가 없으면 홈으로 돌아가고 언어가 한국어로 초기화됩니다.
- **언어**: 한국어 / 영어 / 일본어 / 중국어. 문구는 장소별 구글 시트의 Localization 탭에서 옵니다.

5. 설치 장소와 kioskId

kioskId	장소	레이아웃	카드 단말기
W001	북인사마당	INSADONG	없음
W002	인사동쉼터	INSADONG	없음
W003	남인사마당	NAM_INSADONG	있음
W004	오산 오색시장	OSAN	있음
W005	화성휴게소	HWASEONG	있음
W006	제주공항	JEJU_AIRPORT	있음
W007	제주국제여객터미널	JEJU_AIRPORT	있음
W008	세계자연유산본부	JEJU_HERITAGE	있음
W202	KADA (베트남 하노이)	KADA	없음

기준 파일: `src/shared/config/kioskLocations.ts` (장소별 기능: 카드 단말기, 기부, 좌표, 카메라 회전 등).

kioskId 저장 위치

- 파일: `%APPDATA%\kiosk-app\kiosk-config.json` → `{"kioskId":"W004"}` (베타: `%APPDATA%\kiosk-app-beta\`).
- `src/main/core/KioskConfigStore.ts`가 읽습니다. .env에도, 빌드에도 **들어있지 않기 때문에** 설치 파일 하나로 모든 장소에 설치할 수 있습니다.
- 새 PC에서 설정하기 (한 번 실행 후 앱 재시작):

  ```
  powershell -ExecutionPolicy Bypass -File provision-kiosk.ps1 W004
  ```

  이 스크립트는 설치된 앱 폴더에도 함께 복사됩니다. `-ShopId N`은 상점 API id를 강제로 지정할 때만 사용합니다(보통 생략). 베타 설치는 `-Beta`.

- 설정하지 않은 PC는 `src/shared/constants`의 기본 kioskId를 사용합니다.

6. 데이터: 출처, 가져오는 시점, 저장 위치

데이터	출처	가져오는 시점	저장 위치
화면 문구 / 번역	구글 시트 (Localization 탭)	앱 실행 시마다 + 매일 00:00	SQLite `translations`, 없으면 빌드에 포함된 `*.generated.ts`
궁궐, AI 카테고리, 전국시장, 제주 공항 시설	구글 시트	빌드할 때만 (`npm run sync:sheet`)	`src/renderer/src/data/*.generated.ts`
상점, 관광명소, 배너, 한복 의상, AR 배경, 게임 문제	witteria API	앱 실행 시 + 매일 00:00	SQLite `local_cache`, 이미지는 `remote-images/`
홈 버튼 배치	witteria API	앱 실행 시에만	SQLite
영상 자막	witteria API (제주는 시트로 대체 가능)	앱 실행 시	SQLite
날씨	OpenWeatherMap	30분마다	메모리
환율	한국수출입은행 API	6시간마다	메모리/캐시
항공편 / 여객선 (제주)	공공데이터포털 / KOMSA	2~10분마다	메모리/캐시
터치 통계, 사진 촬영 기록	→ witteria API로 전송	즉시 전송, 실패분은 00:00에 재전송	SQLite `analytics_events`, `failed_requests`
유동인구 수	카메라 → FOOTFALL_API_URL	21:30에 업로드	SQLite `footfall_buckets`
촬영한 사진	카메라 + AI 서버	—	`C:\KioskPhotos`

인터넷이 끊겨도 마지막으로 저장된 데이터로 계속 동작합니다. 화면이 네트워크를 기다리며 멈추는 일은 없습니다.

7. 구글 시트 상세

장소 계열마다 **스프레드시트가 따로** 있습니다: 인사동(W001~W003), 오색(W004), 화성(W005), 제주(W006~W008). KADA는 시트가 없습니다(문구는 `layouts/kada/kadaText.ts`에 있음). 시트 ID는 `GoogleSheetsSyncTransport.ts`(실행 중 사용)와 `scripts/sync-sheet.mjs`(빌드 시 사용) 두 곳에 있으니 항상 같이 수정해야 합니다.

시트를 읽는 두 가지 방식

1. **빌드 시**: `npm run sync:sheet`가 `Localization_Insa`, `PalaceInfo_Insa`, `AICategory_*` 같은 탭을 받아 `src/renderer/src/data/`에 타입이 있는 파일로 만듭니다(원본 CSV는 `sheet-cache/`에 저장). 이 파일들은 앱에 포함되어 오프라인 기본값이 됩니다. 콘텐츠가 바뀌었다면 배포 전에 실행하세요.
2. **실행 중**: `SyncService` + `GoogleSheetsSyncTransport`가 해당 장소의 `Localization_*` 탭을 **앱을 켤 때마다**, 그리고 **매일 밤 00:00**(`NightSyncScheduler.ts`)에 읽어서 SQLite(`translations` 테이블)에 저장합니다. 바뀐 문구는 다음 새로고침 때 반영됩니다.

인증 방식

- 구글 **서비스 계정**을 사용합니다 (`secrets/service-account*.json`, 경로는 `GOOGLE_SERVICE_ACCOUNT_JSON`). 각 스프레드시트는 이 서비스 계정 이메일과 공유되어 있어야 합니다.
- `GOOGLE_SHEETS_ID`는 **켜기/끄기 스위치**일 뿐입니다. 값이 있으면 야간 동기화가 켜지고, 어떤 시트를 읽을지는 kioskId로 정해집니다.

매일 00:00 야간 작업 내용

상점/관광명소/배너/의상/배경/게임 새로고침 → 실패한 통계 재전송 → 시트 문구 다운로드 → 분석 데이터 업로드 → 드라이브 사진 업로드 재시도 → 오래된 키 측정 기록 정리.

8. 키오스크 PC의 파일 위치

무엇	위치
설치된 앱 (+ 포함된 .env, secrets, themes, provision-kiosk.ps1)	설치 폴더 → `resources\`
kioskId	`%APPDATA%\kiosk-app\kiosk-config.json`
앱 설정	`%APPDATA%\kiosk-app\settings.json`
데이터베이스 (캐시된 콘텐츠, 통계, 대기열 전부)	`%APPDATA%\kiosk-app\data\kiosk.db`
로그 (10MB 단위 교체), 문제가 생기면 가장 먼저 확인	`%APPDATA%\kiosk-app\logs\`
다운로드한 CMS 이미지	`%APPDATA%\kiosk-app\remote-images\`
사진 (촬영 + AI 결과)	`C:\KioskPhotos` (또는 `PHOTO_SAVE_DIR`)
대기/안내 영상 (직접 복사, 업데이트해도 유지)	`C:\KioskVideos\<세트>\` (또는 `KIOSK_VIDEOS_DIR`)

베타 빌드는 모든 데이터를 `%APPDATA%\kiosk-app-beta\`에 저장하므로, 같은 PC에 두 버전을 함께 설치할 수 있습니다.

9. 매일 전원 스케줄 (자동 종료)

앱이 **실행될 때마다 스스로** 설정합니다 (`src/main/core/PowerManager.ts`). Windows 작업 스케줄러를 사용하고, 로그인 시 앱이 자동 실행되도록 등록합니다.

대상	스케줄	Windows 작업 이름
W001, W002, W003, W005, W202	매일 **02:00 재부팅**	`KioskAutoRestart`
W004, W006, W007, W008	**22:00 종료**, **08:00 시작**	`KioskShutdownAt10PM`, `KioskStartAt8AM`
베타 빌드	매일 **09:00 재부팅**	`KioskBetaAutoRestart`

> **중요:** Windows는 완전히 꺼진 PC를 켤 수 없습니다. 22:00/08:00 장소는 메인보드 **BIOS → "Power On by RTC Alarm" = 08:00**으로 설정해야 합니다. 설정하지 않으면 아침에 키오스크가 꺼진 채로 있습니다.

저장소 루트의 `SetAutoShutdown*.bat` / `UnsetAutoShutdown*.bat`는 예전 수동 방식이며 더 이상 필요 없습니다. PC에서 확인하려면 *작업 스케줄러*에서 위 작업 이름을 찾으면 됩니다. 제주 시간(22:00/08:00)은 오색시장 시간을 그대로 쓴 것이라 장소별로 확인이 필요합니다.

10. 배포와 자동 업데이트

- `npm run build:win`이 설치 파일을 만들고 **GitHub Releases**에 올립니다. 키오스크는 스스로 업데이트합니다(electron-updater).
- **운영** 키오스크는 일주일에 한 번 확인합니다 (`UPDATE_DAY` / `UPDATE_TIME`, 기본 금요일 17:00, 놓치면 다음 실행 때 확인). **베타** 키오스크는 15분마다 확인하며 베타 릴리스만 받습니다.
- 설치는 키오스크가 사용 중이 아닐 때만 진행됩니다 (사진 카운트다운 / AI 생성 중에는 안 함).

- **"지금 업데이트"**: witteria 관리자 사이트의 버튼입니다. 키오스크가 5분마다 확인하고 바로 업데이트합니다. `docs/UPDATE_COMMAND_API.md` 참고.
- 내 PC의 `.env` 와 `secrets/` 가 **설치 파일 안에 그대로 들어갑니다**. 반드시 올바른 운영용 `.env` 가 있는 PC에서 빌드하세요.
- 자세한 내용: `docs/AUTO_UPDATE.md` .

11. .env 설명

`src/main/core/env.ts` 가 읽습니다. 개발 중에는 프로젝트 루트에서, 설치된 앱에서는 `resources\.env` 에서 읽습니다. 키오스크에서 값을 바꾸면 앱 재시작만 하면 됩니다. **절대 커밋하지 마세요.**

키	의미	필수?
외부 API		
`OPENWEATHER_API_KEY`	날씨 카드 + 제주 예보	필수
`EXIM_API_KEY`	환율 (한국수출입은행)	필수
`DATA_GO_KR_SERVICE_KEY`	공공데이터포털 (제주 항공편)	제주
`KOMSA_SERVICE_KEY`	여객선 운항 정보 (제주항)	제주
`WITTERIA_API_BASE`	메인 백엔드: 상점, 배너, 행사, 통계, 사진 QR 업로드. 운영(PROD) 주소 사용.	필수
`VITE_AR_PROCESS_IMAGE_API_URL` , `VITE_AR_PROCESS_API_URL`	AI 한복 사진 서버 (혼자 / 함께)	포토부스
구글 시트		
`GOOGLE_SHEETS_ID`	시트 동기화 켜기/끄기 스위치 (값 자체는 사용 안 함)	필수
`GOOGLE_SERVICE_ACCOUNT_JSON`	서비스 계정 파일 경로, 예: `secrets/service-account.json`	필수
카드 결제 (단말기가 있는 W003~W008만)		
`PAYMENT_HTTP_HOST` / `_PORT`	로컬 결제 서버 주소 (`127.0.0.1:8080`)	필수
`PAYMENT_CORS_ORIGINS`	결제 서버를 호출할 수 있는 웹페이지: witteria 상점 + Vercel 기부 앱 둘 다	필수
`PAYMENT_CENTRAL_BASE_URL` , `_AUTH_TOKEN` , `_TIMEOUT_MS`	중앙 결제 서버	필수
`PAYMENT_TL3800_*`	카드 단말기(TL-3800): 단말기 ID, 시리얼 포트 설정(포트 비우면 자동 탐색), 재시도 시간	필수
`PAYMENT_OUTBOX_*`	서버 전송에 실패한 결제의 재시도 대기열	기본값 OK
`PAYMENT_LOG_LEVEL`	결제 에이전트 로그 상세도	기본값 OK
업데이트		
`UPDATE_CHANNEL`	`latest` = 운영, `beta` = 테스트 키오스크	필수
`UPDATE_DAY` , `UPDATE_TIME`	주간 업데이트 확인 시점 (기본 금요일 17:00)	선택
`UPDATE_BETA_INTERVAL_MIN`	베타 확인 간격 (기본 15분)	선택

`UPDATE_COMMAND_POLL_MIN`, `UPDATE_COMMAND_API_URL`	"지금 업데이트" 확인 설정	선택
유동인구 카운팅		
`FOOTFALL_ENABLED`, `_LINE`, `_FPS`, `_SCORE`, `_TRACK_SCORE`, `_MIN_BOX`	세부 조정값. 기본값이 현재 운영값. `docs/FOOTFALL.md` 참고	선택
`FOOTFALL_API_URL`, `FOOTFALL_API_KEY`	21:30에 집계를 올릴 주소. 비우면 PC에만 보관.	선택
개발 / PC 설정		
`DEV_MODE`	`true`면 장소 전환 버튼 표시. **실제 키오스크에서는 반드시 false.**	개발
`KIOSK_VIDEOS_DIR`	대기 영상 폴더 (개발 시 내 PC의 영상 폴더)	개발
`PHOTO_SAVE_DIR`	`C:\KioskPhotos` 대신 사용할 폴더	선택

kioskId는 일부러 .env에 넣지 않았습니다 (5장 참고).

12. 자주 하는 작업

하고 싶은 일	방법
화면 문구 수정	해당 장소 구글 시트의 Localization 탭을 수정. 키오스크는 다음 실행 / 자정에 반영. 다음 배포 전에 `npm run sync:sheet`도 실행.
화면 디자인 수정	`src/layouts/<장소>/` (컴포넌트 + `.module.css`). 색상은 `themes/`.
새 장소 추가	`kioskLocations.ts`에 추가 → 레이아웃 선택/제작 → 시트 ID 두 곳에 추가 → PC 프로비저닝.
새 API 데이터 추가	`src/main/services`에 서비스 → `container.ts`에 등록 → `ipc/handlers`에 핸들러 + `shared/ipc`에 채널 → `renderer/src/store`에 스토어 → 레이아웃에서 사용.
DB 변경	새 마이그레이션 파일(`009_…ts`)을 추가. 기존 파일은 절대 수정하지 않기.
새 키오스크 설치	설치 → 한 번 실행 → `provision-kiosk.ps1 W00x` → 영상을 `C:\KioskVideos`에 복사 → BIOS RTC 설정(22:00/08:00 장소) → 재시작.
키오스크 문제 확인	`%APPDATA%\kiosk-app\logs\` 확인. DB는 `scripts/db-inspect.mjs`로 조회.

13. 주의할 점

- `README.md`와 `PROJECT_STATE.md`는 **오래된 문서**입니다. 코드와 이 문서를 기준으로 하세요.
- package.json의 `@nestjs/*`, `serialport`는 포함된 결제 에이전트용이며 앱 자체에서는 쓰지 않습니다.
- 일부 코드 주석에는 야간 동기화가 02:00이라고 되어 있지만 실제로는 **00:00**입니다 (`NightSyncScheduler.ts`).
- 이미 홈 화면이면 무입력 초기화가 홈 화면을 다시 그리지 않습니다. 홈 화면의 팝업은 스스로 닫혀야 합니다 (`useInactivityReset`).
- 카메라 AI(MediaPipe)는 `resources/mediapipe`에 오프라인으로 포함되어 `appres://`로 제공됩니다. 갱신은 `npm run vendor:mediapipe`.
- `.env`, `secrets/`, 사진은 절대 커밋하지 마세요.